

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 1

Roll no: 56 PRN No: 12411305 Name of Student: Anuja Dhiraj Kumbhar

Subject Name : Deep Learning

Problem Statement :

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Algorithm :

1. Start the program.
2. Import TensorFlow, Keras, Matplotlib, and other required libraries.
3. Load the sample dataset (MNIST/CIFAR-10).
4. Split the dataset into training and testing data.
5. Normalize the image pixel values by dividing them by 255.
6. Display a few sample images to understand the dataset.
7. Create a Sequential neural network model.
8. Add the input, hidden, and output layers.
9. Compile the model using the Adam optimizer and appropriate loss function.
10. Train the model using the training data for a few epochs.
11. Test the model on the testing data.
12. Display the accuracy and loss of the model.
13. Stop.

Code :

```
import tensorflow as tf

import matplotlib.pyplot as plt

(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.mnist.load_data()

train_images = train_images /255.0

test_images = test_images / 255.0

print(train_images.shape)

print(test_images.shape)

print(train_labels)

plt.imshow(train_images[1], cmap='gray')

plt.show()

my_model = tf.keras.models.Sequential()

my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))

my_model.add(tf.keras.layers.Dense(128, activation='relu'))

my_model.add(tf.keras.layers.Dense(10, activation='softmax'))

my_model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

my_model.fit(train_images, train_labels, epochs=3)

val_loss, val_acc = my_model.evaluate(test_images, test_labels)

print('Test accuracy: ', val_acc)

my_model.save("my_mnist_model.keras")
```

```
(train_images, train_labels), (test_images, test_labels) =  
tf.keras.datasets.fashion_mnist.load_data()  
  
train_images = train_images / 255.0  
  
test_images = test_images / 255.0  
  
print(train_images.shape)  
  
print(test_images.shape)  
  
print(train_labels)  
  
plt.imshow(train_images[1], cmap='gray')  
  
plt.show()  
  
plt.imshow(train_images[2], cmap='gray')  
  
plt.show()  
  
plt.imshow(train_images[0], cmap='gray')  
  
plt.show()  
  
my_model = tf.keras.models.Sequential()  
  
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))  
  
my_model.add(tf.keras.layers.Dense(128, activation='relu'))  
  
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))  
  
my_model.compile(optimizer='adam',  
loss='sparse_categorical_crossentropy', metrics=['accuracy']) #Adaptive  
Moment Estimation #sparse_categorical_crossentropy for multi-class  
classification  
  
my_model.fit(train_images, train_labels, epochs=3)
```

```
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
```

```
print('Test accuracy: ', val_acc)
```

```
my_model.save("fashion_mnist.keras")
```

Output :

```
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
(train_images, train_labels), (test_images, test_labels) = tf.keras.datas
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-d  
11490434/11490434 ————— 2s 0us/step
```

```
# Scale down values of pixels from 0-255 to 0-1(to reduce the computation)
train_images = train_images / 255.0
test_images = test_images / 255.0
```

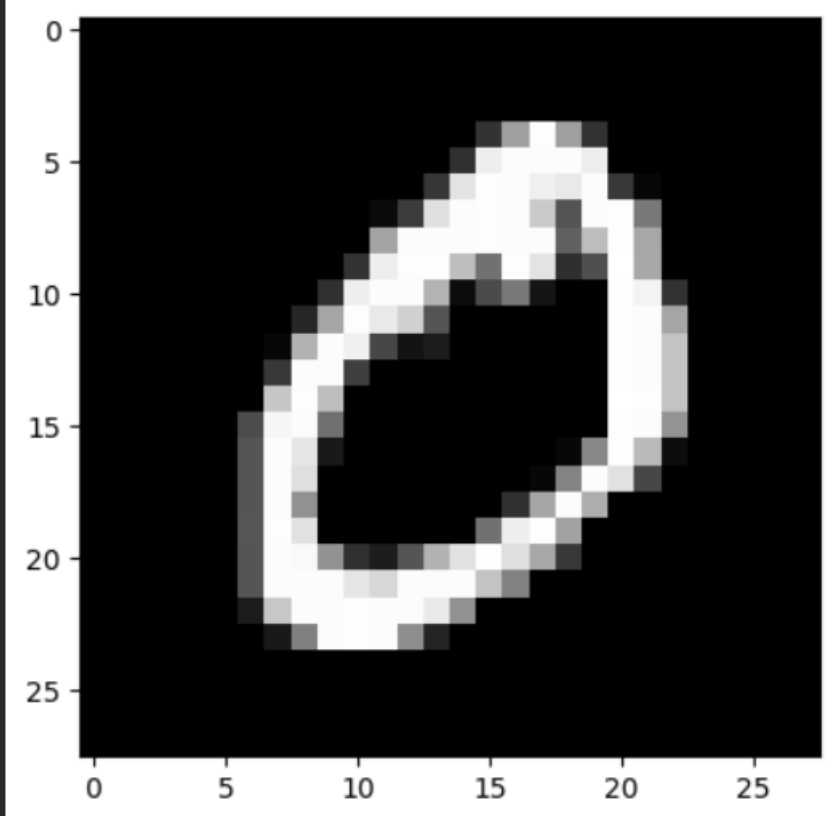
```
print(train_images.shape)
print(test_images.shape)
print(train_labels)
```

```
# Display first image
plt.imshow(train_images[1], cmap='gray')
plt.show()
```

```
.. (60000, 28, 28)
   (10000, 28, 28)
   [5 0 4 ... 5 6 8]
```

[5 0 4 ... 5 6 8]

...



```
# Defining neural network model
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))      #Rectifie
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))    #softmax

# Compiling the model
my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy')

# Training the model(no.of epochs more the more computational cost)
my_model.fit(train_images, train_labels, epochs=3)
```

```
Epoch 1/3
1875/1875 ————— 10s 5ms/step - accuracy: 0.9258 - loss: 0.2
Epoch 2/3
1875/1875 ————— 9s 5ms/step - accuracy: 0.9654 - loss: 0.11
Epoch 3/3
1875/1875 ————— 10s 4ms/step - accuracy: 0.9758 - loss: 0.0
<keras.src.callbacks.history.History at 0x7c2655c39eb0>
```

```
# Checking model accuracy on test data
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)
```

```
313/313 ————— 1s 3ms/step - accuracy: 0.9710 - loss: 0.0927
Test accuracy: 0.9710000157356262
```

```
(train_images, train_labels), (test_images, test_labels) = tf.keras.datas
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-d>
29515/29515 ————— 0s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-d>
26421880/26421880 ————— 2s 0us/step

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-d>
5148/5148 ————— 0s 0us/step

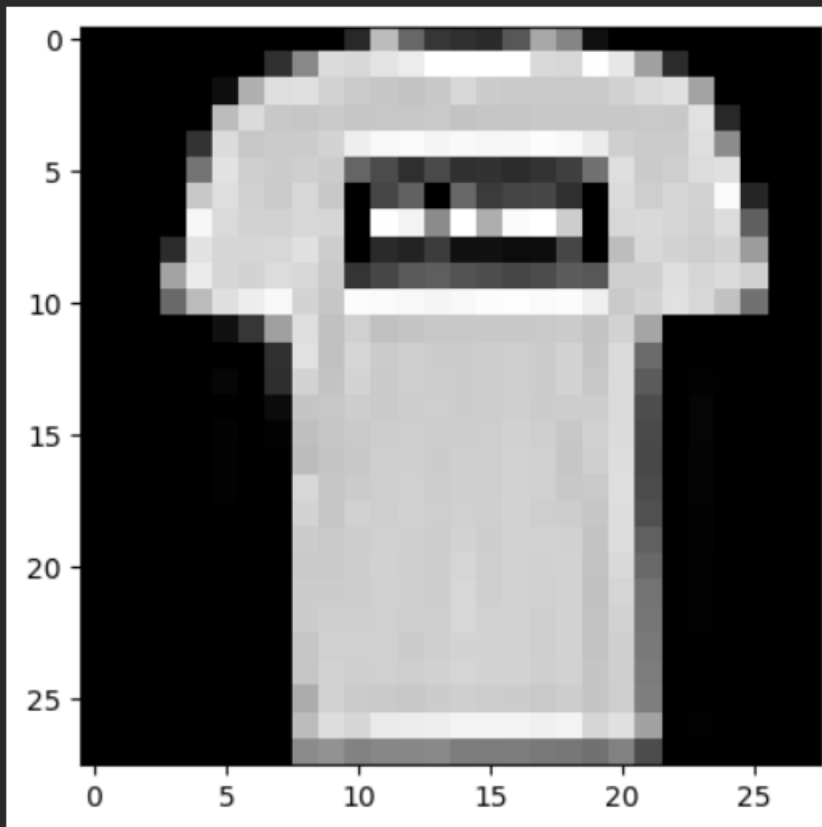
Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-d>
4422102/4422102 ————— 1s 0us/step

```
train_images = train_images / 255.0  
test_images = test_images / 255.0
```

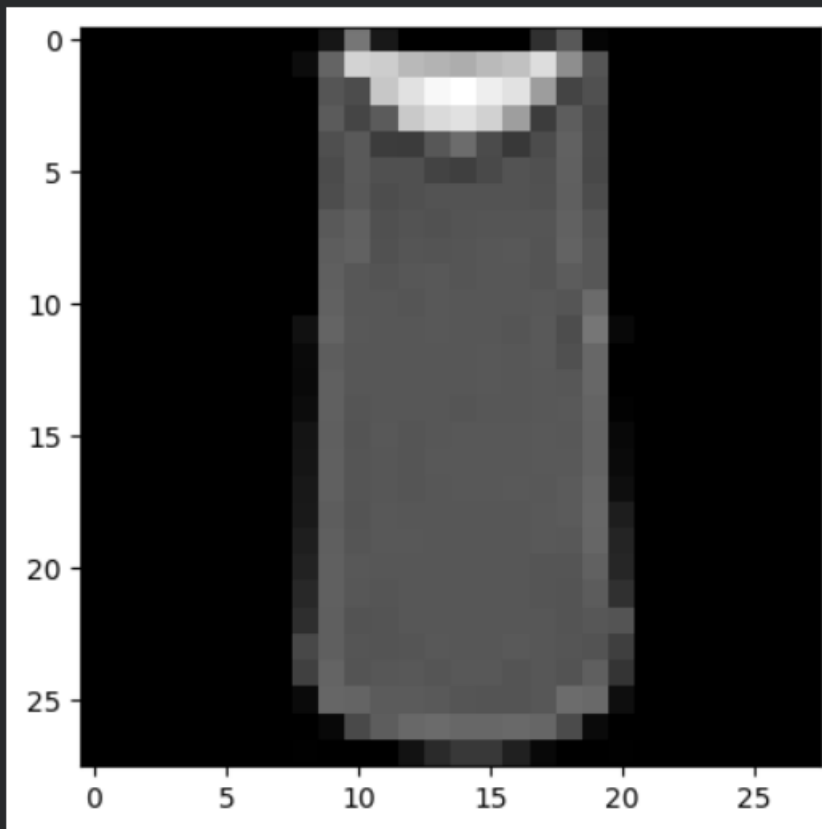
```
print(train_images.shape)  
print(test_images.shape)  
print(train_labels)
```

```
(60000, 28, 28)  
(10000, 28, 28)  
[9 0 0 ... 3 0 5]
```

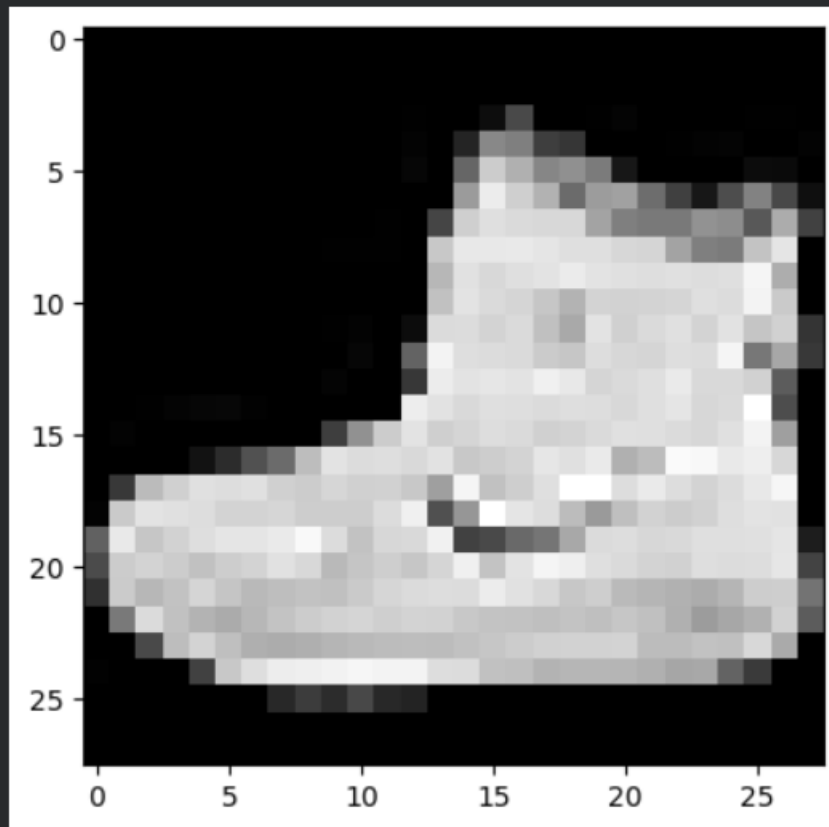
```
plt.imshow(train_images[1], cmap='gray')  
plt.show()
```




```
plt.imshow(train_images[2], cmap='gray')  
plt.show()
```



```
plt.imshow(train_images[0], cmap='gray')  
plt.show()
```



```
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))    #Rectifie
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))  #softmax

# Compiling the model
my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy')

# Training the model(no.of epochs more the more computational cost)
my_model.fit(train_images, train_labels, epochs=3)
```

```
Epoch 1/3
1875/1875 ————— 10s 5ms/step - accuracy: 0.8256 - loss: 0.4
Epoch 2/3
1875/1875 ————— 9s 5ms/step - accuracy: 0.8655 - loss: 0.37
Epoch 3/3
1875/1875 ————— 8s 4ms/step - accuracy: 0.8778 - loss: 0.33
<keras.src.callbacks.history.History at 0x7c2655c47140>
```

```
# Checking model accuracy on test data
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)
```

```
313/313 ————— 1s 3ms/step - accuracy: 0.8735 - loss: 0.3610
Test accuracy: 0.8734999895095825
```